Information Security

From Security Goals to Cryptographic Trust

A textbook treatment of Lectures 1 to 5

Based on the lectures, TPs and quiz of Imane Fouad

October 25, 2026

Contents

How to read this book **3**

1 Foundations of Information Security **4**
1.1 Precision, Properties, Risk 4
1.2 Services and Controls 6
1.3 Actors and Attack Classes 7
1.4 Malware by Mechanism 7
1.5 Domains, Access Control, GRC 9

2 The Web and Stateful Tracking **11**
2.1 The Web in Four Objects 11
2.2 Sessions and the Cookie Attachment Rule 12
2.3 Sites, Origins, Parties 13
2.4 Cross-Site Tracking versus First-Party Analytics 14
2.5 Respawning and the Other Stores 15
2.6 Cookie Syncing and Real-Time Bidding 16
2.7 Clickjacking and Framing 17

3 Stateless Tracking and Browser Defences **19**
3.1 Recognition Without State 19
3.2 Entropy: How Much a Feature Identifies 20
3.3 Attributes and Their Physics 21
3.4 Uniqueness, Linkability, Consistency 22
3.5 Blocking: Filter Lists and Behaviour 23
3.6 Browsers Against Tracking 24

4 Symmetric Cryptography **26**
4.1 Cryptosystems, Kerckhoffs, Attack Models 26
4.2 Classical Ciphers and Their Cryptanalysis 27
4.3 XOR and the One-Time Pad 28
4.4 Confusion, Diffusion, Feistel 29
4.5 DES 29
4.6 AES 30
4.7 Modes of Operation 32
4.8 Key Exchange: Diffie–Hellman 33

5 Public-Key Cryptography, Integrity and Trust **34**
5.1 Why Asymmetric 34
5.2 Number Theory for RSA 34
5.3 RSA 35
5.4 Hash Functions and Passwords 36
5.5 MACs and Digital Signatures 37

5.6 Certificates and PKI 38
5.7 Steganography 39

How to read this book

Each concept in this book is presented in the same rhythm, and every section closes with a *Go deeper* box (the exact details, numbers and edge cases a demanding examiner asks about) followed by *Hard questions, answered.*

1. **Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
2. **Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
3. **Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
4. **Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
5. **Examples.** From "trivially small" to "genuinely useful."
6. **Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, "say it out loud" version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Comfort with binary and hexadecimal, elementary probability, and the modular arithmetic of integers; everything else (finite fields for AES, the number theory behind RSA, Shannon entropy) is built where it is needed.

Chapter 1

Foundations of Information Security

1.1 Precision, Properties, Risk

"Is this circle secure?" has no answer; "if I flood-fill outside it, does colour get in?" does, and the answer depends on whether the boundary is closed. Every security claim must name an asset, a property, an adversary's capability and the control that preserves the property; and the control must be built systematically, since a freehand boundary has gaps. The course's numbers justify three later design choices: $\approx 90\%$ of attacks involve human error (so controls must bound the cost of a mistake), a breach costs on average \$3.86M, and takes 191 days to detect (so detection is not optional).

"Secure" is a relation: secure against what*, for* which asset*, with respect to* which property.

Definition 1.1.1 (Security properties (ISO 27001))**.** *Information security is the preservation of confidentiality, integrity and availability; authenticity, accountability, non-repudiation and reliability can also be involved.* **Confidentiality**: information is not made available or disclosed to unauthorised individuals, entities or processes. **Integrity**: data or resources are not improperly changed, deleted or corrupted; *data* integrity (records unaltered) vs *system* integrity (programs, configurations, processes intact). **Availability**: accessible and usable upon demand by an *authorised* entity. **Authentication**: user or data origin accurately identifiable. **Accountability**: actions traceable to those responsible. **Non-repudiation**: an actor cannot later deny an action to a third party (stronger than authentication, which only convinces the receiver).

Reading the formula

To find the broken property, find the *object* the attacker touched and what the system assumed about it. Airport: Eve forges a boarding pass in her own name for a ticket bought in Alice's name; the pass–record check is skipped; the pass, a trusted document, was altered: *integrity*. Targeted ads on private browsing: *confidentiality*. Platform crash before a deadline: *availability*, no adversary required. The properties are independent: each can fail alone.

Example 1.1.2 (Eve attacks Acme)**.** (1) Phishing yields a valid username/password: confidentiality of the credential, and authentication is under-

mined. (2) Buffer overflow injects code into a root process: system integrity (and the actor's authentication). (3) Database logging disabled: accountability. (4) A new DB user created by editing the user table: integrity of authorisation data, authentication. (5) DDoS takes the server down (availability) while the database is copied (confidentiality). Name properties, not techniques, and name all of them.

Definition 1.1.3 (Risk; threat model)**. Asset**: what we protect. **Threat**: something that can cause damage. **Vulnerability**: an exploitable weakness. **Risk**: the potential that a threat exploits a vulnerability and harms an asset. Jim Anderson: security is "a well-informed sense of assurance that the information risks and controls are in balance". A **threat model** fixes (i) the asset, (ii) the vulnerability assumed and defended against, (iii) the attacker's capabilities and knowledge; only then can a defence be said to *prevent*, *detect*, *respond to* or *recover from* the attack. HTTP: asset = communications incl. credentials; vulnerability = packets in clear on open Wi-Fi; attacker = can intercept a user's or the site's packets; control = HTTPS. Risk treatment has four options: treat, accept, transfer (insurance), avoid.

Key insight

Risk needs both a threat and a vulnerability: the same unpatched service is a risk on the Internet and none on an isolated machine. A control is judged *inside* its model: HTTPS answers interception, not phishing.

Go deeper: the details an examiner asks about

- Quiz 1 Q1–3 are one-property scenarios: read without change (C), alter while accessible (I), DoS (A).
- "Upon demand by an authorised entity": an over-aggressive lockout that refuses legitimate users breaks availability.
- Authentication has two senses in the course: user authentication (access control) and data-origin authentication (MACs, signatures); the ISO wording covers both.
- Adversaries are parameters: a behavioural advertiser, Guccifer 2.0 (leaked DNC documents), and the NSA's own adversary Snowden (the insider).

Hard questions, answered

Q. A stolen backup tape is encrypted. Which properties are affected?
A. Confidentiality preserved (strong cipher, safe key); availability of that copy lost (the organisation's availability depends on other copies); integrity of the original untouched.

Q. Why is non-repudiation not implied by authentication?
A. With a shared secret Bob is convinced Alice sent the message, but Bob could have forged it; a judge cannot distinguish. Non-repudiation needs something only Alice can produce: a signature.

Q. Threat model for a stolen password database?
A. Asset: passwords/accounts (C). Vulnerability: verifiers in the attacker's hands. Attacker: unlimited offline guessing with dictionaries. Control: per-user salt + slow hash (Chapter 5). Out of scope: observing passwords at login.

1.2 Services and Controls

Definition 1.2.1 (Service vs control; the two classifications)**.** A **service** (goal, property) is *what* we want: C, I, A, accountability, authentication. A **control** (mechanism) is *how*: encryption, firewalls, training. By implementation: **physical** (guards, locks, monitoring, environmental, physical intrusion detection), **technical** (logical access control, cryptography, security devices, authentication, network IDS, forensics), **administrative** (policies, standards, procedures, personnel screening, awareness training). By function: **preventive** (stops exploitation: encryption, firewall, MFA), **detective** (warns: IDS, cameras, log review), **corrective** (repairs after detection: restoring a known-good image).

Reading the formula

Every control gets *both* labels. Speed round: MFA = technical/preventive; firewall = technical/preventive; cameras = physical/detective; phishing training = administrative/preventive; restoring backups = technical/corrective; IDS alert = technical/detective. The same word can live in two categories (a motion sensor is physical IDS; a network sensor is technical); one control can serve two functions (a logging firewall); and a control can be wrong for a service (encryption gives C, not A: ransomware is encryption against availability). Depth: controls of different categories so one failure is caught by another.

Go deeper: the details an examiner asks about

- Backup = preparation; the *restore* is the corrective act; classify by the action named.
- The three statistics map to the three functions: human error undermines prevention, 191 days is the absence of detection, $3.86M is what correction limits.

Hard questions, answered

Q. Classify: guard checking badges; morning log review; lockout after 5 failures; password-rotation policy; disaster-recovery site.
A. Physical/preventive; administrative/detective; technical/preventive; administrative/preventive; technical (or physical)/corrective.

Q. One control for accountability and one for non-repudiation, with labels.
A. Protected audit logging: technical/detective. Digital signatures under a PKI: technical/preventive.

1.3 Actors and Attack Classes

A **hacker** is skilled; the *hat* is the intent and legality: **black** (criminal, for gain), **white** (authorised, finds and fixes), **grey** (works with organisations but not necessarily lawfully, e.g. tests without permission). By motive and resources: **state-related** (very high resources, espionage/intrusion), **cybercriminals** (high resources, money), **cyber-terrorists** (destruction), **hacktivists** (ideology, defacements), **script kiddies** (low skill, others' tools, amusement). The threat model needs both axes: script kiddies are defeated by patching; states buy unknown vulnerabilities.

Attack classes and the property each targets: **social engineering** (the person is the vulnerability); **DDoS** (bots in a botnet flood a target for hours to days: availability; hard to block because sources are many and look legitimate); **vulnerability exploits** (OWASP Top 10 lists web classes, Metasploit automates); **unauthorised access** by social engineering, brute force or **password cracking** (hashes already stolen, guesses tested *offline*: login rate limits are irrelevant, only per-guess cost and password entropy matter); **privilege abuse** (an insider, enabled by unnecessary privilege, i.e. no least privilege, and by missing logs, i.e. no accountability); **ransomware** (a payload: encrypt, demand the key's price).

Go deeper: the details an examiner asks about

- Grey hat is defined by method, not alternating intent; a white hat without authorisation is grey by this definition.
- A botnet is the product of a malware campaign and the instrument of an availability attack; the slides link the two.

Hard questions, answered

Q. Why does a six-character password survive online guessing with lockout but fall offline?
A. Online: a few tries per hour. Offline: $26^6 \approx 3 \times 10^8$ hashes computed in under a second against the stolen verifier; the only brake is the hash's per-guess cost.

Q. An admin reads customers' messages "to debug". Class and failed principles?
A. Privilege abuse; least privilege (debugging should not require content) and accountability (reads unlogged).

1.4 Malware by Mechanism

Vectors sit on an axis from *technical* (virus, macro virus, exploit, worm) to *social* (drive-by, clickjacking, phishing, spear phishing, trojan); the axis predicts the control (patching and integrity monitoring vs training and interface design).

Definition 1.4.1 (The families)**. Virus**: infects *programs*; modifies them to carry a copy; executes when the host runs, with the host's permissions (virus = host + execution). Components: infection mechanism (vector), trigger (a condition; a *logic bomb*), payload. **Macro virus**: document scripting (Melissa 1999: Word macro, mailed itself via Outlook; today macros carry warnings

and `.xlsm` extensions). **Worm**: seeks *machines*; each infected host attacks others; spreads by network (web-app upload bugs, SSH dictionary attacks), media (USB autostart) or social means; strategies: *random* scanning (noisy), *hit-list* (precompiled targets, stealthy; Stuxnet), *topological* (data on the victim), *local subnet* (behind NAT/firewall). Morris worm (1988): `fingerd` and `sendmail` bugs plus weak passwords; first CFAA (1986) conviction. **Rootkit**: with root privilege, modifies kernel code/data so the OS lies (`ps` hides a process); *not* an infection method but stealth and continued access; needs root obtained elsewhere; "runs at boot" alone is not a rootkit. **Drive-by download**: browser vulnerabilities install malware on a mere visit (Angler kit: visit, redirect, check, exploit, install). **Clickjacking**: input meant for one action performs another by timing (a dialog appears under the "puppy" 200 ms later) or circumstance (a transparent authorisation page over a game); Chapter 2. **Phishing**: impersonate a trusted source to induce an action (click, open, reply, change a setting, transfer money); *spear* phishing is targeted with facts from reconnaissance; Google Docs 2017: a fake share invitation led to an OAuth consent screen, no malware, the goal was *authorisation*. **Trojan**: the victim runs it (`adorablecat.jpg.exe`, fake ad, "install this plugin"); Zeus/Zbot banking trojan.

Key insight

Viruses infect executables to spread; worms infect machines to spread; trojans infect machines if you run them; rootkits infect kernels to avoid detection. Classical viruses are rare today because their one necessary act, modifying a binary, is cheaply prevented (no write access to program files) and detected (track binary hashes). Ransomware is a payload, not a vector.

Example 1.4.2 (Guess the type)**.** A: outdated Windows service exploited, automatic LAN scanning, 200,000 machines in 150 countries, files encrypted, Bitcoin ransom: a *worm* with a *ransomware* payload. B: a compromised accounting-software update, spreads by exploit *and* stolen credentials, ransom note but paying never recovers files: *supply-chain* delivery, *worm* propagation, *wiper* disguised as ransomware. Classify by mechanism, not by the note on the screen.

Technique: Phishing defences

Users: skepticism, limit what you publish (attacker's background material), inspect links, reach known services yourself. Organisations: training; chain-of-command procedures so an email is never a valid channel for critical requests; email link screening (see all clicks, spot trends, black-hole links at forwarding: detective turned corrective).

Go deeper: the details an examiner asks about

- Rootkit detection must come from outside the lying kernel (boot clean media, compare to known-good images).
- Hit-list = targeted + stealthy; random = fast + loud; the slides pair the words deliberately.

- Consent-screen phishing defeats "inspect the link"; organisation-side controls matter.

Hard questions, answered

Q. Hashing binaries stops viruses but not worms. Why?
A. A virus must modify executables (hash changes); a worm arrives as a new process via a network exploit and touches no existing program. Worm controls: patching, segmentation, credential hygiene.

Q. A program hides its files, survives reboot, came as an email attachment the user opened. Vectors and categories?
A. Phishing delivery of a trojan; persistence is ordinary; hiding files by altering OS answers is a rootkit component, which needed root (an exploit or the user running it as admin).

1.5 Domains, Access Control, GRC

Technical domains: network, web (applications, sessions, input), cryptography (encryption, hashing, signatures, PKI), access control (authentication, authorisation, least privilege), endpoint, cloud. Governance: GRC and privacy. **IAAA**: *identification* (claim an identity), *authentication* (prove it), *authorisation* (what it may do), *accountability* (log and audit). Each fails separately: a badge reader identifies the badge, not the holder (add a PIN); a logged-in user reaching `/admin` is an authorisation failure; a shared account defeats accountability. **GRC**: governance (direction, policies, roles, strategy), risk (identify, assess likelihood and impact, treat/accept/transfer/avoid, monitor), compliance (laws, standards, contracts, audits). Web model: `subdomain.mysite.com/folder/page?id=5`; GET fetches, POST submits.

Go deeper: the details an examiner asks about

- Least privilege lives in authorisation; privilege abuse is its failure.
- References to cite: Whitman & Mattord; Stallings & Brown; Anderson's *Security Engineering*; ISO/IEC 27001/27002; NIST; OWASP.

Hard questions, answered

Q. Map a TLS connection to a bank onto IAAA.
A. Identification: the server name; authentication: certificate chain (server) and password/MFA (user); authorisation: the account's permitted operations; accountability: transaction logs tied to the session.

Q. Cyber-insurance and refusing to store card numbers are which treatments?
A. Transfer; avoid.

Summary: Foundations

- Asset, property, adversary, control: all four or it is not a claim. Properties are independent; find the tampered object.

- Risk = threat × vulnerability on an asset; model before defending; treat/accept/transfer/avoid.
- Service = what, control = how; label controls by implementation *and* function.
- Malware by mechanism: virus (programs), worm (machines), trojan (user runs it), rootkit (kernel lies); social end exploits the person.
- IAAA; GRC.

Chapter 2

The Web and Stateful Tracking

2.1 The Web in Four Objects

Definition 2.1.1 (Privacy)**.** Privacy concerns how information about people is *collected, used, shared and retained*, and a person's ability to *understand* what happens to it, *influence* its use, and *avoid* unwanted observation. A tracking mechanism "threatens privacy" when it defeats one of these verbs (invisible: understand; no opt-out: influence; survives deletion: avoid). Cases: Cambridge Analytica (Wylie, 18 March 2018); Google's $392M settlement with 40 US states (November 2022) over location tracking.

Definition 2.1.2 (URL, HTTP, HTML, browser)**.** In `https://informationsecurity.com:443/courses/web-security?week=5&topic=xss` the parts are protocol, hostname (resolved by DNS), port, path, query. HTTP is application-layer, client–server, request (URL, method GET/POST, headers, optional body) / response (status code, resource), and *stateless*: nothing is kept once the TCP connection closes. HTML is declarative structure; JavaScript is the logic. The browser loads, renders (parses HTML, runs scripts), and reacts to events (`onclick`, `onload`, `setTimeout`). `location.href` is the full URL including the fragment (never sent in HTTP), `host` includes the port, `hostname` does not, `search` is the query. History: Bush's Memex 1945, Berners-Lee's ENQUIRE 1980 (local), the 1989 proposal that became the WWW.

Key insight

Two facts drive the chapter. (1) Statelessness forces applications to add state on top of HTTP; trackers reuse that mechanism, so tracking is the web's own design pointed at another goal. (2) A script runs with the authority of the *page that includes it*, not the server that served it: `analytics.js` fetched from `metrics.test` executes as `shop.test`'s code.

Hard questions, answered

Q. `https://a.example/p#secret` includes `<script src="https://t.example/x.js">`. What does `t.example` learn from the request, and the script afterwards?

A. Request: t.example's own cookies, Referer `https://a.example/p` (no fragment), IP, headers. The script, running as `a.example`, can read `location.href` incl. `#secret` and a.example's non-HttpOnly cookies, and send them explicitly.

2.2 Sessions and the Cookie Attachment Rule

A session token in the URL (`?sessionid=8F4A92C1`) leaks through the `Referer` header to every embedded third party, through shared links, logs and history. Cookies (1994) fix the leak and create everything else.

Definition 2.2.1 (Cookie; attachment rule)**.** A server stores a name–value pair with `Set-Cookie`; the browser returns it in `Cookie` on *every request to that domain, regardless of which site initiated the request.* Without `Expires`/`Max-Age` it is a *session cookie*, deleted when the browser closes; with one it is *persistent*.

Reading the formula

Both clauses are load-bearing. *To that domain*: no site can read or receive another's cookie, so identifiers must be smuggled through URLs to cross sites (§2.6). *Regardless of initiating site*: `bbc.com` embedding an image from `doubleclick.net` causes a request carrying doubleclick's cookie; that is third-party tracking and also CSRF. `SameSite` (1990s rule, 2016 attribute) qualifies exactly this clause. Model: a jar indexed by domain; on each request the browser attaches matching cookies without knowing which page caused the request.

Definition 2.2.2 (Attributes)**.** `Domain`: default current host only; set to a parent (`Domain=example.com` from `websec.example.com`) to cover it and all subdomains; cannot name an unrelated domain or a public suffix. `Path`: that path and sub-paths. `HttpOnly`: invisible to `document.cookie`; still *sent*. `Secure`: sent only over HTTPS and settable only from HTTPS. `SameSite=Strict`: never on cross-site requests. `Lax` (default): on cross-site requests only if *top-level navigation and* a *safe method* (GET, HEAD, OPTIONS, TRACE; RFC 7231). `None`: always, requires `Secure` (Chrome rejects None without Secure; Firefox/Edge warn).

Technique: Is cookie c attached to a request from page P to destination D?

(1) D in c's domain scope? (2) Path prefix? (3) If Secure, HTTPS? (4) Same site (registrable domain) for P and D? If yes, attach. If cross-site: Strict no; Lax only for top-level *and* GET-like; None yes. `HttpOnly` never enters: it only governs script reads.

Worked example: Quiz 1, Q4

`https://shop.example.test` sets `aid=A72; Domain=example.test; Path=/; Secure; HttpOnly; SameSite=Lax`. A: from the shop, load `https://cdn.example.test/banner.png`: subdomain covered, HTTPS, same site: *yes*. B: same over `http://`: Secure fails: *no*. C: from `https://social.test` Alice *clicks a link* to `https://shop.example.test/profile`: cross-site but top-level GET: Lax allows: *yes*. D: `social.test` *embeds* `https://shop.example.test/tracker.png`: cross-site subresource, not top-level: *no*. Answer A, C; HttpOnly was a distractor.

Gotcha

Lax needs *both* conditions: a cross-site iframe/image fails top-level; a cross-site POST form is top-level but unsafe. Only a plain link passes. HttpOnly protects against script theft (XSS), not against sending (CSRF, tracking); Secure against network theft; SameSite against cross-site attachment.

Go deeper: the details an examiner asks about

- `Set-Cookie: test=1; Domain=.example.com; Secure; HttpOnly; SameSite=None`: sent on any HTTPS request to example.com and subdomains, cross-site too, invisible to JS, and still a *session* cookie (no expiry).
- Scope is by *name*: `lab.test:8000` and `127.0.0.1:8000` are different jars for the same Flask server (TP).
- Trackers need `None; Secure`; banks prefer Strict (no CSRF at all) or Lax (usability: inbound links arrive logged in).

Hard questions, answered

Q. A bank cookie is `HttpOnly; Secure`, no SameSite. An attacker page auto-POSTs a transfer form. Sent?
A. Under the Lax default: no (unsafe method). Treated as None: yes, transfer succeeds as the victim. HttpOnly/Secure are irrelevant to attachment.

Q. Can `a.example.com` set a cookie `b.example.com` receives? One `example.org` receives?
A. Yes with `Domain=example.com`; no.

2.3 Sites, Origins, Parties

Definition 2.3.1 (eTLD+1; origin)**.** The registrable domain (**eTLD+1**) is the public suffix (`com`, `co.uk`, `github.io`) plus one label: the unit of *ownership*, read from the right. A request is **first-party** if its destination has the address bar's registrable domain, **third-party** otherwise; cookies are first/third-party accordingly. An **origin** is (scheme, host, port); the **same-origin policy** restricts what a script may *read* (documents, storage, `document.cookie`) to its own origin. Subdomains are same-site but cross-origin.

Reading the formula

Site governs *attachment* (SameSite, partitioning); origin governs *access*. The SOP never prevents the browser from *sending* a cookie (Quiz 1 Q6C is false); it prevents `bbc.com`'s script from *reading* doubleclick's cookie. Visiting `https://shop.example.com`: `www`, `images`, `api.shop`, `analytics.example.com` are first party; `example.org`, `example.com.tracker.net` (owner: `tracker.net`), `cdn.google.com`, `shop-example.com` are third party. A tracker needs three capabilities: (1) an identity in the browser (cookies, headers, storages: *stateful*; or recomputed from properties: *stateless*, Chapter 3); (2) a channel back (automatic cookie attachment, or explicit JavaScript); (3) context (Referer, `location`, a parameter).

Hard questions, answered

Q. Visiting `https://news.bbc.co.uk`: classify `static.bbc.co.uk`, `bbc.com`, `bbc.co.uk.cdn-edge.net`.
A. First; third (different registrable domain); third (owner `cdn-edge.net`).

Q. Can a script on `a.example.com` read `b.example.com`'s cookies? Are `Domain=example.com` cookies sent to both?
A. No (cross-origin); yes (same-site, Domain covers both).

2.4 Cross-Site Tracking versus First-Party Analytics

Theorem 2.4.1 (One third-party cookie)**.** *A tracker embedded on sites $\mathcal{S}$ with a persistent cookie learns, per browser, the timed sequence of visits (site and page) to every site in $\mathcal{S}$, with no cooperation from the sites.*

Proof. Each visit to $s \in \mathcal{S}$ triggers a request to the tracker ("regardless of initiating site"); it carries the tracker's cookie ("to that domain"), the same value on every site because it belongs to the tracker's domain, not to s; Referer or an embedding parameter supplies s and the page. Group the log by cookie value. □

First-party analytics looks similar and is not: `accuweather.com` includes `google-analytics.com/script.js`; the script runs as accuweather and sets `ga_id=123` *under accuweather.com*, a first-party cookie; it must send it explicitly (`track?ga_id=123&site=accuweather.com`) since the browser never attaches accuweather's cookie to a google-analytics request. On `nytimes.com` the same script creates a *different* cookie under nytimes. Google gets per-site identifiers it cannot link: within-site repeat visits only.

Key insight

The difference is *under which domain the cookie is stored*: tracker's domain = one identifier everywhere, attached automatically; publisher's domain = one identifier per publisher, sent explicitly. "Third-party cookies blocked" blocks cookies set in third-party context, not third-party *scripts* setting first-party cookies.

Worked example: Quiz 1, Q5–Q7

Q5 (non-HttpOnly first-party `prefs`; `metrics.test/analytics.js`; iframe from `tracker.test`): the script runs in the shop's origin and reads `prefs` (A), and can send it explicitly (D); the iframe is another origin (B false); the request for `analytics.js` carries metrics.test's cookies, not the shop's (C false). *Q6* (pixel from `tracker.test`, `tid; SameSite=None; Secure`, 3P allowed): same `tid` from both publishers (A), tracker links them (D); publisher JS cannot read it (B false); SOP does not govern sending (C false). *Q7* (3P blocked; same script; `aid=N31` on news, `S84` on shop): first-party identifiers (A); not sent automatically to metrics.test (B false); same script ≠ same identifier (C false); script can read and send the current one (D). Merging N31 and S84 needs an external link (login, fingerprint).

Go deeper: the details an examiner asks about

- Publishers set `HttpOnly` on their session cookie partly against the third-party scripts they include.
- Citations: Roesner et al. (NSDI 2012) classified tracking behaviours; Englehardt & Narayanan (CCS 2016, OpenWPM, 1M sites) measured third-party presence.

Hard questions, answered

Q. Minimal change turning an analytics vendor into a cross-site tracker?
A. Have the script load a resource from the vendor's domain that sets a vendor-domain cookie with `SameSite=None; Secure`; it is then attached on every site that includes the script. Under the Lax default the embedded request would not carry it.

2.5 Respawning and the Other Stores

Definition 2.5.1 (Respawning)**.** Recreating a deleted cookie from a copy held in another store (a *zombie* cookie): HTML5 `localStorage` (per origin, no expiry, across sessions), Flash local shared objects (across sessions *and* browsers, since the plugin's store was per machine user), HTTP cache headers `ETag`/`Last-Modified`. Evercookie combines many. All of these can also track *without* cookies.

Mechanics. Tracker iframe on `site1.com` writes `id=789` as cookie and into `tracker.com`'s localStorage. User deletes cookies; on `site2.com` the server issues `id=555`, the script finds 789 in localStorage and reports `old_id=555, new_id=789`; profiles merge. ETag: first response `ETag: "x234dff"`; the browser sends `If-None-Match: "x234dff"` on the next request and may get `304 Not Modified`; nothing forces the tag to describe content, so it can be a user identifier echoed even in private browsing with cookies blocked, as long as the cache lives (KissMetrics, August 2011: `_km_cid` cookie equal to the ETag). Each store has its own scope and its own clearing action; a privacy action is sufficient only if it clears every store the tracker used.

Worked example: Quiz 1, Q10–Q12

Tests: (2) delete cookie, cookie back as 1234 with localStorage 1234: *consistent with* localStorage restore (A); does not *prove* fingerprinting (B false). (3) delete cookie + localStorage (now "new"), cookie still 1234, ETag 1234: consistent with ETag restore (C); profile still 3, so deletion did not break the link (D false). (4) clear cookies, storage, cache: new 5678 everywhere (stores *were* cleared, Q11C false) but fingerprint 88 and profile 3 persist: consistent with stateless relinking (B); a fingerprint identifies a browser, not a named person (A false); shared fingerprints give false positives (D). Q12: clear cookies, local storage and cache *together* (A); the other options leave a source.

Hard questions, answered

Q. Design a respawn surviving "clear site data" but not "clear cache".
A. Identifier in the `ETag` of an always-loaded resource, echoed in `If-None-Match`; site-data clearing removes cookies and storage, not the HTTP cache.

Q. Everything cleared, profile continues. Remaining explanation?
A. A stateless identifier recomputed from browser properties: Chapter 3.

2.6 Cookie Syncing and Real-Time Bidding

Theorem 2.4.1 is bounded by $\mathcal{S}$: doubleclick sees bbc and nytimes, adnxs sees bbc and jumia, each with its own identifier, and neither can read the other's cookie.

Definition 2.6.1 (Cookie syncing)**.** The process by which two trackers link the identifiers each assigned to the same browser. Mechanism: the browser requests a pixel from tracker 1 with `Cookie: id=1234`; tracker 1 replies `302 Location: https://tracker2.com/sync?t1=1234`; the browser follows, and the request to tracker 2 carries *tracker 2's* cookie `id=5678` plus the URL parameter; tracker 2 records (5678 ↔ 1234) and may redirect back with `t2=5678`. The identifier crossed domains *inside the URL*, the one place the attachment rule does not police; neither tracker read the other's cookie.

RTB: the **SSP** offers a publisher's ad space and forwards each opportunity to the **ad exchange**, which runs the auction and picks the winner; **DSPs** evaluate and bid for advertisers. The bid request (page URL, approximate location, pseudonymous ID) is *broadcast to all DSPs*; an SSP syncs its cookie with a DSP by redirect so the DSP recognises the user in the request. Only part of the process is visible in the browser.

Worked example: Quiz 1, Q8, Q9, Q16

Q8: `GET tracker-a/sync, Cookie: aid=A42` → `302 tracker-b/sync?aid=A42` → `GET ..., Cookie: bid=B71`: B associates A42 with B71 (A); A42 is a URL parameter, not B's cookie (B false); A passed its ID in the redirected URL (C); no shared origin needed (D false). *Q9*: the SSP helps the publisher sell (A); the losing DSP-X still received the request (B); not only the winner (C false); the pseudonymous ID is stable, it enables linking (D false). *Q16* (drawing): browser in the middle with

both jars; request 1 carrying 1234; the 302 with 1234 in the URL; request 2 carrying 5678 and the parameter; the mapping table at tracker 2; optional return redirect.

Hard questions, answered

Q. Does per-site cookie partitioning (Firefox, Chapter 3) stop a sync between two trackers on the *same* publisher?
A. No: both have cookies in that site's jar and the redirect links them there. Partitioning stops reuse across publishers.

Q. Why a redirect rather than a script?
A. Only a request to tracker 2's domain carries tracker 2's cookie; the redirect produces such a request while carrying tracker 1's ID in the URL. No script can read either cookie across origins.

2.7 Clickjacking and Framing

TP1: `game.html` embeds `<iframe src="portal.html">` positioned over a "Catch Me" button, higher `z-index`, `opacity: 0`; at score 4 the button is moved under the portal's Login; Alice's click lands on the invisible, *authenticated* Login. Relies on: clicks go to the top-most element and `opacity: 0` keeps the frame in hit-testing (`visibility: hidden` or `pointer-events: none` would not); the framed page received its cookies ("regardless of initiating site"; a Lax/Strict session cookie would not arrive, which mitigates); nothing forbids framing unless the portal says so.

Definition 2.7.1 (Framing controls)**.** Response headers sent by the *framed* page. `X-Frame-Options: DENY` | `SAMEORIGIN` (`ALLOW-FROM` obsolete). CSP `frame-ancestors 'none'` | `'self'` | list of origins (`frame-ancestors 'self' https://apps.um6p.ma`); takes precedence over X-Frame-Options; the only way to whitelist a trusted third origin.

Hard questions, answered

Q. The portal must never be framed, except later by one trusted university app. Headers?
A. Now: `frame-ancestors 'none'` + `X-Frame-Options: DENY` for old browsers. Later: `frame-ancestors 'self' https://trusted.um6p.ma`; X-Frame-Options cannot express it.

Q. Setting anti-framing headers on the *game* page: useful?
A. No; the attacker controls the game page. The victim page must send them.

Summary: Stateful tracking

- Cookies are attached to every request to their domain, whoever initiated it; SameSite qualifies the second clause (Lax = top-level *and* safe method; None needs Secure).
- Site (eTLD+1) governs attachment; origin governs script access.

- One tracker-domain cookie links all embedding sites; publisher-domain analytics cookies cannot.
- Respawn from localStorage, Flash, ETag; clear all stores together; what survives everything is stateless.
- Syncing crosses domains inside a redirect URL; RTB broadcasts bid requests.
- Clickjacking: transparent authenticated iframe; defence: X-Frame-Options / CSP frame-ancestors on the framed page.

Chapter 3

Stateless Tracking and Browser Defences

3.1 Recognition Without State

Definition 3.1.1 (Fingerprint; stateful vs stateless)**.** A **browser fingerprint** is the set of information about a device, from hardware through OS to browser configuration, that a page can observe; **fingerprinting** is collecting it to recognise the device. **Stateful** tracking stores an identifier and reads it back (cookies, storage, advertising IDs): "I recognise you because I stored something". **Stateless** tracking recomputes a recognisable pattern from properties exposed *now*: "I recognise you from what you reveal each time". Clearing state does not stop recomputation.

Reading the formula

The test is *where the identifier comes from.* An ETag is stateful (it sits in the cache) though it is not a cookie; a canvas hash is stateless though it looks like an ID. The TP splits collection into *passive* (what the server receives with no code: IP, `User-Agent`, `Accept-*`, client hints), *active* (JavaScript queries APIs: `navigator`, `screen`, `window`, `Intl`, `document.referrer`) and *behavioural* (typing speed, key-hold time, inter-key interval, pauses, corrections; probabilistic, since keyboard, fatigue and stress change them). Features in a fixed order, hashed with SHA-256, give one short identifier (`/collect?fp=8b91c7a4...`) that the user cannot read, which is why fingerprinting is hard to notice.

The attack surface grew with the browser: Netscape Communicator 5.0 (1998, ≈742k lines) to Chromium 120 (January 2024, ≈31M lines); each new API (GPU, sensors, USB, Bluetooth, AR/VR) is a feature and a fingerprint attribute. A 1995 page saw "Netscape, French"; a 2024 page sees OS, screen, time zone, GPU model, audio and video capabilities.

Go deeper: the details an examiner asks about

- A fingerprint identifies a browser instance, never a named person (Quiz 1 Q11A false); exact matches have false positives.
- Quiz 1 Q13: before any JavaScript runs, the server observes IP, User-

Agent, Accept-Language, Accept-Encoding and client hints directly (A); screen and fonts need active collection (B false); headers do distinguish browsers (C false); no cookie is needed (D false).

- Disabling JavaScript stops active and behavioural collection; passive headers and stateful mechanisms remain.

Hard questions, answered

Q. Why is a hashed fingerprint not a permanent identifier?
A. Any changed feature (browser update alters the User-Agent, a new font) changes the hash; tracking over time needs *linking* algorithms (§3.4), not exact matching.

3.2 Entropy: How Much a Feature Identifies

Definition 3.2.1 (Surprisal and Shannon entropy)**.** If a feature takes value x_i with probability $P(x_i)$, observing x_i yields $-\log_2 P(x_i)$ bits of information (its *surprisal*). The expected information over the population is the entropy

$$H(X) = -\sum_i P(x_i) \log_2 P(x_i).$$

Low entropy: many users share the value; high entropy: the value is rare and narrows the population more.

Reading the formula

A value shared by half the population gives 1 bit (halves the candidates); one shared by 1/1000 gives ≈ 10 bits. Independent features add: $H(X,Y) = H(X)+H(Y)$. Singling out one of 7×10^9 devices needs $\log_2(7\times10^9) \approx 33$ bits. A uniform feature over n values has $\log_2 n$ bits (the maximum); a constant feature has 0. The distribution matters as much as the number of values: a skewed feature identifies less than a balanced one with the same support.

Worked example: Quiz 1, Q15 and a variant

100 fingerprints: 50 Windows, 25 Linux, 25 macOS.

$$H = 0.5\log_2 2 + 0.25\log_2 4 + 0.25\log_2 4 = 0.5 + 0.5 + 0.5 = 1.5 \text{ bits}$$

(answer C; the maximum for 3 values is $\log_2 3 \approx 1.585$). Time zone 80% UTC+1, 10% UTC+0, 10% UTC+2: $H = 0.8 \times 0.322 + 2 \times 0.1 \times 3.322 = 0.258 + 0.664 = 0.92$ bits: three values, less identifying than the OS, because skewed.

Hard questions, answered

Q. A feature has 16 equally likely values; another has 1000 values but one of them covers 99% of users. Which identifies more?
A. First: $\log_2 16 = 4$ bits. Second: $0.99\log_2(1/0.99) + 0.01\log_2(999/0.01) \approx 0.014+0.166 \approx 0.18$ bits on average (the rare values are informative but rarely

seen). Entropy is an expectation over the population.

Q. Why do fingerprint studies report entropy per attribute rather than "number of distinct values"?
A. Because identification power is the information observed, which depends on how values are distributed; 1000 values with one dominant is nearly useless, 16 balanced values are strong. Independence is needed to add attribute entropies; correlated attributes (OS and fonts) add less than their sum.

3.3 Attributes and Their Physics

- **HTTP headers / User-Agent**: passive; grew from `NCSA_Mosaic/2.0 (Windows 3.1)` to long strings naming engine, OS and version.
- **Screen, platform, time zone, hardware concurrency, fonts, extensions**: active; e.g. `3840x2160x24`, `Win32`, UTC+8, 4 cores.
- **Canvas** (Mowery & Shacham, W2SP 2012): JavaScript draws hidden text and shapes; the *instructions* are identical but the *rendered pixels* differ with browser, OS, fonts, GPU and graphics stack (anti-aliasing, hinting); read pixels, hash, use the hash as an attribute. Quiz 1 Q14: render predefined content, read pixels, hash (B); it stores nothing, retrieves no images, reads no serial number.
- **WebGL**: vendor and renderer strings ("NVIDIA, GeForce GTX 4070 Ti").
- **GPU timing** (DrawnApart, Laor et al., NDSS 2022): two machines identical in model, OS, browser and GPU model still differ because manufacturing variation makes some GPU execution units slightly faster; JavaScript measures timing traces; a deep-learning model compresses a trace into an embedding; same device gives a similar embedding despite noise. On 2,550 devices and 370,392 fingerprints over 7 months in the wild, adding it to FP-Stalker raised the median tracking duration (7-day revisit setting) from 17.5 to 28 days (+10.5, $\approx 60\%$). The only *hardware-level* attribute on the syllabus; it improves linkability, not uniqueness.
- **Battery Status API** (2012; Olejnik, Acar, Castelluccia, Diaz, *The Leaking Battery*, 2015): level, charging state, time remaining. Site A sees "73%, 14,237 s, not charging"; 20 seconds later site B sees "73%, 14,210 s": few devices match, and the drift is predictable, so the visits link *without any stored identifier*; the link expires as the battery drifts. A temporary signal for short-term linking; also abusable for price discrimination.
- **IP address**: passive; WWW'20: 87% of devices kept the same IPv4 for 30+ days (5,443 devices, 41,566 addresses), country/ISP dependent, the device–IP graph 93% unique; 20% show home–work cycles of 30+ days. Shared by NAT, so weak alone, strong combined.
- **Mobile**: also highly unique (S&P'16), vendor/model dependent, different attributes (User-Agent, emoji rendering, canvas).

Hard questions, answered

Q. Two identical laptops: which attribute separates them and why?
A. DrawnApart: GPU execution-unit timing reflects manufacturing variation,

invisible to every software attribute.

Q. Explain Battery-API linking and why it is not an identifier.
A. Level plus time-remaining plus charging state at time t is rare; at $t + 20$ s the same device shows the same level and ≈ 20 s less; probable match. Minutes later the combination has drifted and matches many devices: it links visits close in time only.

Q. Canvas needs JavaScript; User-Agent does not. Why?
A. Canvas requires executing drawing code and reading pixels back (active); the User-Agent is a header sent with every request (passive).

3.4 Uniqueness, Linkability, Consistency

Three research questions. **Uniqueness**: Panopticlick, 470,161 fingerprints, 94.2% unique; *Beauty and the Beast* (S&P'16), 118,934, 89.4%; *Hiding in the Crowd* (WWW'18), 1,816,776 desktop fingerprints, only 35.7% unique. **Linkability**, how fingerprints that *change* (updates, new fonts) can still be followed: FP-Stalker (S&P'18) tracks users for 55 days on average with the right linking algorithm; DrawnApart extends it; Blink (SEAMS'15) breaks it by generating a random disposable environment (OS, browser, fonts) per session; FPRandom (ESSoS 2017) adds controlled noise to canvas (adopted by Brave). **Consistency**: FP-Scanner (USENIX Security'18) shows most fingerprint-altering tools are detectable because attributes stop agreeing (User-Agent says Windows, fonts, platform and WebGL say Linux), and some tools leave per-user traces; the defence becomes a signal. Other works on the map: Fingerprinting in Style (USENIX Sec'21), Taming the Shape Shifter and Web Runner 2049 (DIMVA'20), Morellian analysis (DIMVA'19), FP-Redemption (DIMVA'21). `amiunique.org` (November 2014) shows your own fingerprint and its evolution.

Interpretation & discussion

Why did uniqueness fall from 94% to 36%? A far larger and more diverse sample; the disappearance of high-entropy attributes (Flash, Java, plugin lists); homogeneous populations on identical OS/browser versions. The conclusion is not "tracking stopped": 36% unique still leaves most browsers in small groups that an IP, a login or a battery signal collapses to one, and linkability over time does not need global uniqueness. Fingerprinting for *security* uses the same data defensively: "does this login look familiar?" compares the current browser to past logins of a known account; the fingerprint is a *risk signal*, never proof of identity (false positives and drift), so a mismatch triggers step-up authentication rather than denial.

Go deeper: the details an examiner asks about

- Quiz 1 Q11: Test 4 (all stores cleared, fingerprint 88 and profile 3 persist) is *consistent with* stateless relinking (B), and different browsers may share 88 (D); it proves neither a named identity (A) nor that clearing failed (C).
- Naive randomisation attacks linkability but creates *inconsistency*; a tool used by few people is itself a rare attribute. Brave's site-dependent noise

(§3.6) is the answer.

Hard questions, answered

Q. A defence randomises canvas on every load. Which property does it attack and what can go wrong?
A. Linkability (and the canvas attribute's uniqueness). Risks: FP-Scanner-style coherence checks detect the tool; the tool's rarity becomes an identifying attribute; sites that need stable canvas break.

Q. Why is 35.7% unique still a privacy problem?
A. Non-unique fingerprints lie in small groups; add IP or a short-term signal and the group is one device; and FP-Stalker-style linking follows a browser for weeks without uniqueness.

3.5 Blocking: Filter Lists and Behaviour

Definition 3.5.1 (Filter list; EasyList syntax)**.** A maintained set of rules describing requests or page elements a blocker should block, allow, redirect or hide: Disconnect (Firefox, Brave), EasyList (ads) and EasyPrivacy (tracking) (AdBlock Plus, uBlock Origin). A network rule `||tracker.com^` blocks every request whose URL matches: `||` anchors at the start of a domain (so `sub.tracker.com` matches, `nottracker.com` does not), `^` marks the end of the domain or a separator such as `/`. A rule beginning `@@` is an *exception* that allows something a broad rule would break.

Filter lists are *knowledge-based*: "we already know this domain tracks". Limitations: *coverage* (unlisted trackers load), *evasion* (rotating domains and URLs, first-party and CNAME techniques), *breakage* (login, video, payment, embeds), *maintenance* (continuous measurement and updates). **Behaviour-based** protection asks what a third party *does*: Privacy Badger (EFF) watches for identifying cookies, local-storage "supercookies" and fingerprinting, and restricts a third party seen tracking across several unrelated sites (the threshold trades false positives on ubiquitous CDNs against coverage). FP-detect inverts the problem: it detects which lists or extensions are active, so the blocker itself becomes an attribute.

Worked example: EasyList exercise

With `||amazon-adsystem.com^` present, both `/ad.js` and `/widgets/q?item=10` on that domain match the domain rule and are blocked *unless* an `@@` exception (e.g. for `/widgets/`) exists; the exercise's point is that exceptions exist to prevent breakage, so one checks the list rather than guesses. Rule "block all of `ads.example.net` except `/consent/`": `||ads.example.net^` plus the exception `@@||ads.example.net/consent/`.

Hard questions, answered

Q. A tracker is served from `static.publisher.com`. Which limitation, which protection?
A. Evasion by first-party/CNAME hosting; URL-based lists miss it. DNS-level inspection (Firefox resolves the CNAME chain) or behaviour-based de-

tection if it tracks across sites (it usually appears on one, so even Privacy Badger may miss it).

3.6 Browsers Against Tracking

- **Third-party cookie deprecation**. Google announced the end of third-party cookies in Chrome (January 2020): planned Q2 2022, delayed to Q4 2023, then Q4 2024; in 2024 Chrome moved to a *user-choice* approach; in 2025–2026 Google confirmed no universal phase-out. Lesson: browser privacy is shaped by compatibility, advertising economics, standards and deployment, not only by technique. Advertisers meanwhile built their own user IDs, device data and first-party data pipelines, and every platform opened a Conversions API (Meta, Amazon, Microsoft Offline Conversions, Google Enhanced Conversions, TikTok Events, X) to enrich first-party data.
- **Privacy Sandbox** (Chromium): Topics API (interest-based targeting), Protected Audiences (retargeting), Shared Storage (frequency capping), Attribution Reporting. *Topics*: each week the browser derives 5 topics from browsing history and adds one random topic, from a taxonomy of 469; the random topic gives deniability. Apple and Mozilla objected; PETS'24: the *sequence* of weekly topic sets observed by several sites is specific enough to serve as a quasi-identifier (stateless tracking built from a privacy API), and the design is "at odds with the natural diversity of user behaviours".
- **Firefox Total Cookie Protection**: a separate cookie jar per *top-level site*; a tracker embedded on A and B gets two different cookies, so Theorem 2.4.1 fails (it degrades to per-site analytics). Targets stateful cross-site tracking only; fingerprinting needs other defences; syncing within one site still works.
- **Brave randomisation**: subtle, *site-dependent* noise on selected fingerprintable outputs ("farbling"): the same site sees stable values (nothing breaks), different sites see different values (no linking); cites PriVaricator and FPRandom. Contrast FP-Scanner: inconsistent spoofing (one attribute changed, others untouched) reveals the tool and sometimes the real OS.
- **CNAME cloaking**: DNS `f0Ab1ze.mywebsite.com CNAME bad.tracker.com`; the browser sees a first-party hostname, cookies are first-party, URL filter lists miss it; only resolving the CNAME chain (Firefox exposes DNS-level APIs to blockers) reveals the tracker; a threat to transparency since the mapping lives server-side, and regulators (CNIL) must audit companies to see it.

Go deeper: the details an examiner asks about

- Dates to know: 2020 announce; 2022 Q2, 2023 Q4, 2024 Q4 planned; 2024 user choice; 2025–26 no phase-out.
- Quiz 1 Q6 under Total Cookie Protection flips: (A) and (D) become false, two jars, two `tid` values.
- Defence philosophies: isolate state (Firefox) vs randomise outputs (Brave) vs block known/behaving trackers (lists, Badger); a browser needs all three.

Hard questions, answered

Q. Why can Topics approximate an identifier although each week's set is only 5 of 469 topics plus a random one?
A. Several sites observe the same browser's weekly sets over many weeks; the sequence of sets is highly specific to one browsing history, so the sites can link the browser by matching sequences; the random topic adds noise but not enough across weeks.

Q. Why is CNAME cloaking "a big threat for transparency"?
A. Nothing in the page reveals the tracker: the hostname is first-party, the alias is in DNS and server configuration; users and URL-based blockers see a first-party request; only DNS inspection or an audit of the company exposes it.

Q. Compare Firefox partitioning and Brave randomisation: which threat does each address and what does each miss?
A. Firefox: stateful cross-site tracking via cookie reuse; misses fingerprinting, CNAME-cloaked first-party cookies, same-site syncing. Brave: fingerprint stability; misses stateful tracking unless combined with blocking/partitioning.

Summary: Stateless tracking and defences

- Stateless = recompute from exposed properties; passive / active / behavioural; a hash makes it one unreadable identifier of a *browser*, not a person.
- $H = -\sum P \log_2 P$; independent features add; 33 bits single out a device; skew lowers entropy.
- Attributes: headers, screen/fonts/time zone, canvas rendering, WebGL, GPU timing (hardware), battery (short-term), IP (stable weeks).
- Uniqueness fell to 36% on desktop; linkability (55 days, 28 with GPU) is the real threat; inconsistent spoofing is detectable.
- Lists (`||domain^`, `@@`) are knowledge-based and evadable (CNAME); Badger is behaviour-based; Firefox partitions, Brave randomises per site; Topics is a quasi-identifier; Chrome kept third-party cookies.

Chapter 4

Symmetric Cryptography

4.1 Cryptosystems, Kerckhoffs, Attack Models

Definition 4.1.1 (Cryptosystem)**.** *Cryptology* = *cryptography* (secret writing: hiding meaning) + *cryptanalysis* (breaking it). A cryptosystem is (M, C, K, E, D) with $c = E(m, k_e)$ and $m = D(c, k_d)$; *symmetric* if $k_e = k_d$ (everyone who knows k knows the whole secret), *asymmetric* if $k_e \neq k_d$ and k_d is infeasible to derive from k_e. The **key space** is the set of possible keys; **brute force** tries them all. Cryptography provides *confidentiality* (against eavesdroppers), *integrity* (content unaltered) and *authentication* (origin as claimed); ciphers alone give only the first.

Key insight

Kerckhoffs's principle: the system must remain secure when everything except the key is public. Consequences: algorithms are published and reviewed (DES and AES were open competitions); a compromised key is replaced without redesign; "secret algorithm" is a red flag because algorithms leak and cannot be rotated. Modern practice: public, sophisticated algorithm, short secret key (the inverse of the historical "simple algorithm, long key").

Definition 4.1.2 (Attack models)**.** What the cryptanalyst has besides the algorithm and the target ciphertext: **ciphertext only** (COA); **known plaintext** (KPA: some plaintext–ciphertext pairs under the key); **chosen plaintext** (CPA: plaintexts of the attacker's choice encrypted); **chosen ciphertext** (CCA: ciphertexts of the attacker's choice decrypted); **chosen text** (both). Power increases COA < KPA < CPA, CCA; security under a stronger model implies the weaker. Ways to attack: *analytical* (cleverness: weaknesses in the algorithm or its use), *brute force* (money: parallel search; *computationally secure* = cost to crack exceeds the value), *social engineering*, *implementation attacks* (software, hardware, side channels).

Hard questions, answered

Q. Give a realistic setting for each attack model.
A. COA: intercepted radio. KPA: encrypted files with known headers (PDF magic bytes). CPA: a web app that encrypts a cookie value the attacker controls. CCA: a server that decrypts submitted tokens and reveals whether

padding was valid.

Q. Classify: power-trace analysis of a smart card; bribing an operator; trying all 2^{56} DES keys; exploiting ECB block independence.
A. Implementation; social engineering; brute force; analytical.

4.2 Classical Ciphers and Their Cryptanalysis

Definition 4.2.1 (Caesar, substitution, Vigenère)**. Caesar**: $c = (p + k) \bmod 26$; key space 26. **Simple substitution**: a fixed permutation of the alphabet; key space $26! \approx 4.03 \times 10^{26}$. **Vigenère**: a keyword $k_1 \dots k_L$ repeats; letter i is Caesar-shifted by $k_{i \bmod L}$.

Theorem 4.2.2 (Why substitution falls to frequency analysis)**.** *A monoalphabetic substitution permutes the letter-frequency histogram of the plaintext but does not change its shape; the ciphertext letter with frequency $\approx 12.7\%$ is the image of E, and so on. Hence the key is recovered letter by letter from statistics, in time independent of the key-space size.*

Proof. Each plaintext letter a maps to one fixed ciphertext letter $\sigma(a)$, so $\#\{i : c_i = \sigma(a)\} = \#\{i : p_i = a\}$: the multiset of counts is preserved and only relabelled. English counts are far from uniform (E 12.7, T 9.1, A 8.2, O 7.5, I 7.0, N 6.7, S 6.3, H 6.1, R 6.0…; J, X, Q, Z rarest), so the relabelling is largely determined by sorting; short words, doubled letters and repeated patterns resolve ties. □

Technique: Breaking Vigenère with known or guessed key length L

Split the ciphertext into L groups `text[i::L]`, $i = 0..L-1$; every letter in group i was shifted by the same k_i, so each group is a Caesar cipher (Theorem 4.2.2 applies within the group). In each group take the most frequent letter, assume it is E, shift = (that letter − E) mod 26, key letter = the shift (E→G: shift 2, key C). If L is unknown: repeated trigrams occur at distances that are multiples of L (Kasiski), or compare the index of coincidence for candidate L. The attack needs the key to *repeat*; a random key as long as the message and never reused leaves one letter per group and no statistics (the one-time pad).

Worked example: TP, warm-up and Room 1

Caesar $K = 4$: `CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ` → `GVCTXS MW JYR YRXMP XLI TVSJIWWSV WECW UYMD`. $K = 8$ decrypt: `GWCL JMBBMZ VWB JM CAQVO BWWTA BW AWTDM BPQA` → `YOUD BETTER NOT BE USING TOOLS TO SOLVE THIS`. Unknown key, brute force 26 shifts: `ESP VPJ TD FYVYZHY ECJ MCFEP QZCNP` is shift 11: `THE KEY IS UNKNOWN TRY BRUTE FORCE`. Substitution table (A→Q, B→M, C→J, D→Z, E→T, …): `KEEP THE SECRET SAFE` → `LTTN EKT YTJRTE YQGT`; `ERVYE DXVR SXFPJ` → `TRUST YOUR LOGIC`. Slide: `GSRH RH Z HVXIVG` is the reversed alphabet (Atbash): `THIS IS A SECRET`.

Go deeper: the details an examiner asks about

- Key-space size is necessary, not sufficient: 26! keys and minutes to break. The exam question "is a large key space enough?" wants Theorem 4.2.2.
- Enigma: German naval advantage early in WWII; broken by Turing's *bombe*; the lesson the slides draw is that with computers substitution and transposition alone are obsolete, and modern ciphers *combine* substitution with repeated transposition so that statistical relations vanish (DES, AES).
- "Computationally secure" is economic (cost > value); "unconditionally secure" (one-time pad) is information-theoretic.

Hard questions, answered

Q. Why is substitution with 26! keys broken faster than Caesar with 26 keys by brute force?
A. Analytical attack vs exhaustive search: the statistics survive the permutation; a few dozen informed guesses beat 26 trial decryptions only in the sense that both are trivial, but the point is that 26! gave no security at all.

Q. Two messages XOR-encrypted with the same one-time key. What does Eve get?
A. $c_1 \oplus c_2 = m_1 \oplus m_2$; with structured text she recovers both (crib dragging). Reuse destroys the pad.

4.3 XOR and the One-Time Pad

XOR: "different → 1, same → 0"; $(x \oplus k) \oplus k = x$, so the same operation encrypts and decrypts. `Hello` ⊕ repeating key `key` gives `00100011 00000000 00010101 00000111 00001010`. With a key that is random, as long as the message, and never reused, this is the *one-time pad*: each ciphertext is equally likely under every plaintext of that length, so COA learns nothing. With a *repeating* key it is Vigenère on bytes and breaks under every model: KPA gives plaintext ⊕ ciphertext = key; CPA with all-zero plaintext returns the key; CCA with all-zero ciphertext returns the key; COA sees that the second ciphertext byte is `00000000`, hence the second key byte equals the second plaintext byte, and continues from language structure.

Hard questions, answered

Q. Why must modern ciphers resist *chosen*-plaintext attacks rather than just known-plaintext?
A. Attackers routinely control inputs (they can make a server encrypt their data); a scheme that leaks under CPA is broken in practice even if KPA is safe, and CPA security implies the weaker models.

4.4 Confusion, Diffusion, Feistel

Definition 4.4.1 (Confusion and diffusion (Shannon))**. Confusion**: make the relation between key, plaintext and ciphertext complex (nonlinear); Caesar's $c = (p + k) \bmod 26$ has none. **Diffusion**: spread the influence of each input bit over many output bits so that local statistics vanish. Block ciphers achieve them by rounds of substitution (confusion) and permutation/mixing (diffusion).

Theorem 4.4.2 (Feistel networks invert for free)**.** *Let a round map* $(L_{i-1}, R_{i-1}) \mapsto (L_i, R_i)$ *with* $L_i = R_{i-1}$, $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$ *for* any *function* f*. Then* (L_{i-1}, R_{i-1}) *is recovered by* $R_{i-1} = L_i$, $L_{i-1} = R_i \oplus f(L_i, K_i)$*. Hence an* n*-round Feistel cipher is decrypted by the same algorithm with the round keys in reverse order, and* f *need not be invertible.*

Proof. L_i is a copy of R_{i-1}; substituting into R_i and XOR-ing $f(R_{i-1}, K_i)$ once more cancels it since $x \oplus y \oplus y = x$. Apply round by round from $i = n$ down to 1. □

4.5 DES

Definition 4.5.1 (DES)**.** Block 64 bits, key 56 bits (the 64-bit input key has 8 parity bits at positions 8, 16, …, 64, dropped by *permuted choice* PC-1), 16 Feistel rounds with distinct 48-bit subkeys, an initial permutation IP before and its inverse after. The round function $f(R, K_i)$, 32 bits in, 32 out: (1) **expansion** E: 32 → 48 bits by duplicating the two boundary bits of each 4-bit group; (2) XOR with K_i; (3) eight **S-boxes** $S_1..S_8$, each 6 bits → 4: *row* = outer bits b_1b_6, *column* = inner bits $b_2b_3b_4b_5$; the only nonlinear step; (4) **permutation** P: a fixed rearrangement that sends each S-box's output bits to different S-boxes next round (diffusion). IP, E and P are position tables: they move bits, never change values.

Worked example: One round, slide and toy

Slide: input `0x0123456789ABCDEF`; IP's first row (58, 50, 42, 34, 26, 18, 10, 2) selects bits giving first output byte `11001100`; R_0 = `0xF0AAF0AA`; $E(R_0)$ = `011110 100001 010101 010101 011110 100001 010101 010101`; after $\oplus K_1$: `0x6117BA866527`; first group `011000`: row 00 = 0, column 1100 = 12, $S_1[0, 12] = 5 =$ `0101`; all boxes: `0x5C82B597`; P: `0x234AA9BB`. Toy ($L_0 =$ `1010`, $R_0 =$ `1001`, $E' = [4, 1, 2, 3, 4, 1]$, $K_1 =$ `101011`, $P' = [2, 4, 3, 1]$): $E'(R_0) =$ `110011`, $\oplus K_1 =$ `011000`, $S_1 \to$ `0101`, $P' \to$ `1100`, $L_1 = R_0 =$ `1001`, $R_1 = L_0 \oplus$ `1100` = `0110`.

Definition 4.5.2 (Key schedule and decryption)**.** PC-1 keeps 56 bits, split into 28-bit halves C_0, D_0. Round i: rotate each half left by 1 bit in rounds 1, 2, 9, 16 and by 2 bits otherwise; PC-2 selects 48 of the 56 as K_i (each K_i is a permuted subset of the key). Total rotation $4 \cdot 1 + 12 \cdot 2 = 28$, so $C_{16} = C_0$, $D_{16} = D_0$. Decryption (Theorem 4.4.2): the same 16 keys in order $K_{16} \dots K_1$; equivalently run the schedule with no rotation in round 1, right rotations of 1 in rounds 2, 9, 16 and of 2 elsewhere. No new keys are computed. Toy:

key `10101101`, PC-1 $= [1,2,3,5,6,7] \to$ `101110`, $C_0 =$ `101`, $D_0 =$ `110`; round 1 shift 1: `011`, `101`; PC-2 $= [1,2,4,5]$ on `011101` $\to K_1 =$ `0110`; round 2 continues from C_1, D_1: `110`, `011` $\to$ `110011` $\to K_2 =$ `1101`.

Interpretation & discussion

Strength: $2^{56} \approx 7.2 \times 10^{16}$ keys; at 10^9 keys/s, 7.2×10^7 s ≈ 2.3 years on one machine; the EFF's \$250,000 machine (1998) cracked DES in under 2 days. DES died of key size, not of cryptanalysis. **3DES** applies DES three times (EDE, so $K_1 = K_2 = K_3$ degrades to DES for compatibility) with 2 keys (112 bits effective) or 3 (168); double DES is pointless because *meet in the middle* (encrypt a known plaintext under all K_1, decrypt its ciphertext under all K_2, match) costs $\approx 2^{57}$, not 2^{112}. 3DES survives in legacy finance (ATMs); AES replaced it (and is 5× faster in CPU hardware).

Go deeper: the details an examiner asks about

- Why DES decrypts with reversed keys and AES does not: Theorem 4.4.2 needs the Feistel structure; AES's operations act on the whole block and need explicit inverses.
- S-box indexing: "outside bits = row, inside bits = column"; row 2 of S_1 is 4 1 14 8 13 6 2 11 15 12 9 7 3 10 5 0.
- Cipher table: DES 1975 (56/64), 3DES 1995 (112/168, 64), Twofish, Serpent, Rijndael 1998 (128/192/256, 128); AES standardised 2001 (FIPS-197).

Hard questions, answered

Q. S_1 on input `100110`?
A. Row $b_1b_6 =$ `10` $= 2$, column $b_2..b_5 =$ `0011` $= 3$: $S_1[2,3] = 8 =$ `1000`.

Q. Why is the effective key 56 bits when 64 are entered, and why does 2DES give ≈ 57 bits?
A. Eight parity bits are discarded by PC-1. Meet in the middle turns the 2^{112} search into two 2^{56} tables matched in the middle.

4.6 AES

Definition 4.6.1 (GF(2^8) arithmetic)**.** Bytes are polynomials of degree < 8 over $\{0,1\}$; addition is XOR; multiplication is polynomial multiplication reduced modulo $m(x) = x^8 + x^4 + x^3 + x + 1$ (`0x11B`). Multiplying by x (`02`, `xtime`): shift left one bit, and if the bit shifted out was 1, XOR with `1B` (that is $x^8 \equiv x^4 + x^3 + x + 1$). $03 \cdot b = (02 \cdot b) \oplus b$. Every nonzero byte has an inverse, which is what makes the S-box and MixColumns invertible.

Definition 4.6.2 (AES-128)**.** Rijndael (Rijmen, Daemen; FIPS-197, 2001); block 128 bits as a 4×4 byte *state* filled column-wise (b_{i+4j} at row i, column j); keys 128/192/256 with 10/12/14 rounds and $N_r + 1$ round keys; a substitution–permutation network, *not* Feistel. Encryption: AddRoundKey(K_0); rounds

1..$N_r - 1$ of **SubBytes** (each byte through the S-box; confusion), **ShiftRows** (row r rotated left by r; moves bytes *between* columns), **MixColumns** (each column multiplied by $\begin{smallmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{smallmatrix}$ in GF(2^8); mixes *within* a column), **AddRoundKey** (XOR with K_i); final round without MixColumns. ShiftRows and MixColumns together give diffusion: after two rounds every byte depends on every input byte. Decryption: AddRoundKey(K_{10}); for rounds 9..1: InvShiftRows (right rotations), InvSubBytes (inverse S-box), AddRoundKey, InvMixColumns (matrix with `0E 0B 0D 09`); then InvShiftRows, InvSubBytes, AddRoundKey(K_0). Key expansion: 44 words; $w_0..w_3$ = key; for $i \equiv 0 \pmod 4$: RotWord, SubWord, XOR `Rcon`; always XOR with w_{i-4}; Rcon first bytes `01 02 04 08 10 20 40 80 1B 36` (powers of 02 in the field, hence `1B` after `80`).

Worked example: MixColumns, first byte of column `D4 BF 5D 30`

$02 \cdot$ D4: `11010100` $\rightarrow$ `10101000` = `A8`, carry out, $\oplus$`1B` = `B3`. $02 \cdot$ BF: `10111111` $\rightarrow$ `01111110` = `7E`, carry, $\oplus$`1B` = `65`; $03 \cdot$ BF = $65 \oplus$ BF = DA. b_0 = B3 $\oplus$ DA $\oplus$ 5D $\oplus$ 30 = 69 $\oplus$ 5D $\oplus$ 30 = 34 $\oplus$ 30 = 04. Whole column $\rightarrow$ `04 66 81 E5`; AddRoundKey: `04`$\oplus$`A0` = `A4`. SubBytes examples: `19`$\rightarrow$`D4`, `00`$\rightarrow$`63`, `03`$\rightarrow$`7B`, `71`$\rightarrow$`A3`. ShiftRows of the index state: row 1 `11 12 13 10`, row 2 `22 23 20 21`, row 3 `33 30 31 32`. AddRoundKey with all-`0F` key: XOR each byte with `0F` (`23`$\rightarrow$`2C`).

From the TP: AES without diffusion

Two rounds on one byte, $c = S(S(p \oplus k_0) \oplus k_1) \oplus k_2$, with ShiftRows and MixColumns removed. From $(00, \text{A4})$: $k_2 = \text{A4} \oplus S(S(k_0) \oplus k_1)$, a function of (k_0, k_1); enumerate the 2^{16} pairs, derive k_2, keep those consistent with $(01, \text{F8})$ and $(02, \text{3F})$. Each byte position is attacked *independently* with three known plaintexts; with diffusion the state must be attacked as a whole (2^{128}). Confusion without diffusion is attackable byte by byte.

Go deeper: the details an examiner asks about

- The final round omits MixColumns because a trailing public linear map adds no security and would spoil the symmetry between encryption and decryption.
- SubBytes is the only nonlinear step; without it AES is a linear map solvable by linear algebra.
- AES-128 vs 256: key length, rounds (10 vs 14), round keys (11 vs 15), expansion details; block size and the four operations unchanged.

Hard questions, answered

Q. Compute $02 \cdot 57$ and $03 \cdot 57$.
A. `57`=`01010111` $\rightarrow$ `10101110` = `AE`, no carry: `AE`; $03 \cdot 57$ = AE $\oplus$ 57 = F9.

Q. Which step gives confusion, which diffusion, what breaks without each?
A. SubBytes: confusion (remove: linear, trivial). ShiftRows+MixColumns: diffusion (remove: byte-wise 2^{16} attack). AddRoundKey: the key (remove:

no secret).

4.7 Modes of Operation

Definition 4.7.1 (ECB, CBC, CTR)**.** A block cipher handles one block; a *mode* extends it. **ECB**: $C_i = E_K(P_i)$; parallel both ways; *deterministic per block*, so equal plaintext blocks give equal ciphertext blocks and the *structure* leaks (the penguin). **CBC**: $C_i = E_K(P_i \oplus C_{i-1})$, $C_0 = IV$; $P_i = D_K(C_i) \oplus C_{i-1}$; encryption sequential, decryption parallel with random access; the **IV** is random, *not secret* (sent in clear) and prevents a repeated first block from repeating its ciphertext. **CTR**: $C_i = P_i \oplus E_K(\text{nonce}\|i)$; a stream cipher from a block cipher; fully parallel; the nonce must never repeat under a key (else a two-time pad). Others: CFB, OFB, CCM (CTR for confidentiality + CBC-MAC for integrity: authenticated encryption).

Theorem 4.7.2 (Error propagation in CBC)**.** *A single flipped bit in ciphertext block C_i garbles plaintext block P_i entirely and flips exactly that bit in P_{i+1}; later blocks are unaffected.*

Proof. $P_i = D_K(C_i) \oplus C_{i-1}$ passes the corrupted block through D_K (pseudorandom output). $P_{i+1} = D_K(C_{i+1}) \oplus C_i$ XORs the corrupted C_i directly. P_{i+2} uses C_{i+1}, C_{i+2} only. □

From the TP: OpenSSL mission 2, ECB on an image

OpenSSL's salted format: 8-byte signature `Salted__` + 8-byte *salt* (public; the key is derived from password and salt by PBKDF2), so skip 16 bytes. The image is $432 \times 475 = 205{,}200$ bytes $= 12{,}825$ blocks exactly, so PKCS#7 adds one full padding block: ciphertext 205,216, file 205,232 bytes. Counting equal 16-byte blocks (`collections.Counter`) and colouring equal blocks alike reproduces the picture's outline: a *pattern, not the secret*. `openssl enc -d -aes-256-ecb -pbkdf2 -p -in classified.enc -out recovered.raw`; re-encrypting with `-aes-256-cbc` leaves essentially no repeated blocks because each block is XOR-ed with the previous ciphertext first.

Hard questions, answered

Q. Why is the IV sent in clear not a weakness, and what would be?
A. The IV only needs to be unique/unpredictable so equal first blocks differ; knowing C_0 does not help invert E_K. Reusing an IV, or a predictable one under CPA, is the weakness.

Q. Rank ECB, CBC, CTR: encryption parallel / decryption parallel / random access / hides patterns.
A. ECB yes/yes/yes/no; CBC no/yes/yes/yes; CTR yes/yes/yes/yes (unique nonce).

Q. Strong cipher, weak system: AES-256-ECB on 16-byte customer IDs.
A. Equal IDs give equal ciphertexts; frequency and equality analysis works without touching AES. Use CBC/CTR with fresh IVs or authenticated encryption.

4.8 Key Exchange: Diffie–Hellman

Symmetric ciphers need the key delivered over a *secure channel*. Merkle (1974, idea rejected by his professor; 1975 paper rejected by CACM as "extremely dangerous to transmit key information in the clear") and Diffie–Hellman (1976) showed it is not needed. Public g, p; Alice picks a, sends $A = g^a \bmod p$; Bob picks b, sends $B = g^b \bmod p$; both compute $g^{ab} = B^a = A^b \bmod p$. Eve sees g, p, A, B and must solve the **discrete logarithm** (slide: $6^x \bmod 13 = 2$, i.e. $x = 5$; $6^x \bmod 13 = 9$, $x = 4$), trivial for $p = 13$, infeasible for 2048-bit p. DH gives a shared secret, not authentication: a *man in the middle* negotiates k_1 with Alice and k_2 with Bob and relays everything; endpoints must be authenticated (Chapter 5).

Hard questions, answered

Q. Which hard problem underlies DH, and which underlies RSA?
A. Discrete logarithm; integer factorisation.

Q. Does encrypting the DH messages with Bob's public key stop the MITM?
A. Only if Alice holds Bob's *authentic* public key; if Eve can substitute hers, she decrypts and relays. The problem is key authenticity: certificates.

Summary: Symmetric cryptography

- Kerckhoffs: only the key is secret. Models COA < KPA < CPA/CCA.
- Substitution dies to frequency analysis (statistics survive relabelling); Vigenère splits into L Caesar groups; a non-repeating random key is the one-time pad.
- Confusion (nonlinear S-boxes) + diffusion (permutation/mixing); Feistel inverts for free with reversed keys.
- DES: 64/56, 16 rounds, E–XOR–S-box–P, rotations 1,2,9,16; brute-forced in 1998; 2DES falls to meet-in-the-middle.
- AES: GF(2^8) with `1B`; SubBytes, ShiftRows, MixColumns, AddRoundKey; 10/12/14 rounds; SPN needs explicit inverses.
- ECB leaks structure; CBC chains with a public random IV; CTR is a parallel stream; CCM adds integrity.
- DH: discrete log; no authentication, hence MITM.

Chapter 5

Public-Key Cryptography, Integrity and Trust

5.1 Why Asymmetric

With symmetric keys, N people need $\binom{N}{2} = N(N-1)/2$ keys (6 people: 15) and a secure channel for each; with key pairs, $2N$ (6 people: 12), and no pre-shared secret. The mailbox principle: anyone can drop a letter, only the owner can open the box. Diffie, Hellman and Merkle published the idea in 1976.

Definition 5.1.1 (One-way and trapdoor functions; requirements)**.** A **one-way function** $y = f(x)$ is easy to compute and infeasible to invert. A **trapdoor function** is one-way except to whoever holds secret information t (generated with f by an algorithm Gen); the public key is f, the private key is t. A public-key system must make key generation, encryption with the public key and decryption with the private key *easy*, and deriving the private key from the public key, or the message from the ciphertext, *infeasible*. Candidate: factoring; $6895601 = 1931 \times 3571$ is hard to find and trivial to check given $t = 1931$.

Key insight

Public keys remove the need for a *secret* channel but create the need for an *authentic* one: Alice must know that the public key is Bob's, or a man in the middle substitutes his own. Certificates (§5.6) exist for this.

5.2 Number Theory for RSA

Definition 5.2.1 (Modular arithmetic, φ, inverses)**.** $a \equiv b \pmod{n}$ iff $n \mid a-b$. $\gcd(a,n) = 1$ iff a has a multiplicative inverse mod n, found by the extended Euclidean algorithm (for tiny numbers: try $d = 1, 2, \ldots$ until $ed \bmod \varphi = 1$). Euler's totient $\varphi(n)$ counts $1 \le a < n$ coprime to n; for distinct primes $\varphi(pq) = (p-1)(q-1)$. **Euler's theorem**: $\gcd(a,n) = 1 \Rightarrow a^{\varphi(n)} \equiv 1 \pmod{n}$. Fast exponentiation: square repeatedly and multiply the needed powers (the slide's $26^2, 26^4$, then $26^7 = 26^4 \cdot 26^2 \cdot 26$).

5.3 RSA

Definition 5.3.1 (RSA (Rivest, Shamir, Adleman, 1977))**.** Choose distinct large primes p, q (random, similar size); $n = pq$; $\varphi = (p-1)(q-1)$; choose e with $1 < e < \varphi$, $\gcd(e, \varphi) = 1$; compute d with $ed \equiv 1 \pmod{\varphi}$. Public key (n, e), private key (n, d); p, q, φ stay secret. Encrypt $c = m^e \bmod n$; decrypt $m = c^d \bmod n$.

Theorem 5.3.2 (Correctness)**.** *For* $0 \le m < n$, $(m^e)^d \equiv m \pmod{n}$.

Proof. $ed = 1 + k\varphi$ for some integer k. If $\gcd(m, n) = 1$, Euler gives $m^{ed} = m \cdot (m^{\varphi})^k \equiv m$. If $p \mid m$, both sides are $0 \bmod p$, and mod q Fermat gives $m^{ed} \equiv m \cdot (m^{q-1})^{k(p-1)} \equiv m$; by the Chinese remainder theorem the congruence holds mod n (symmetrically for $q \mid m$). □

Worked example: Slide keys and the TP password

Slide: $p = 3, q = 11$, $n = 33$, $\varphi = 20$, $e = 3$, $d = 7$ $(21 \equiv 1)$; $m = 4$: $c = 64 \bmod 33 = 31$, $31^7 \bmod 33 = 4$; $m = 5$: $c = 125 \bmod 33 = 26$, $26^2 \equiv 16$, $26^4 \equiv 25$, $26^7 \equiv 25 \cdot 16 \cdot 26 \equiv 5$. TP: $p = 5, q = 11, e = 3, C = 23$: $n = 55$, $\varphi = 40$, $d = 27$ $(81 = 2 \cdot 40 + 1)$; $23^2 \equiv 34$, $23^4 \equiv 34^2 = 1156 \equiv 1$, so $23^{27} = (23^4)^6 \cdot 23^3 \equiv 23 \cdot 34 = 782 \equiv 12$: password $m = 12$; check $12^3 = 1728 = 31 \cdot 55 + 23$. Fresh pair: $p = 7, q = 13, e = 5$: $n = 91, \varphi = 72, d = 29$; $m = 2 \mapsto c = 32$.

Interpretation & discussion

Security rests on factoring n: with p, q one computes φ and d. The RSA Factoring Challenge shows the frontier: RSA-768 (232 digits) factored December 2009, RSA-250 (829 bits) February 2020; 1024-bit keys "are out there", 2048 is the default. A 50-bit modulus (TP) falls to trial division up to $\sqrt{n} \approx 2^{25}$ in seconds; a 512-bit one needs the number field sieve (deprecated, but not trial division). RSA is slow and the message must be smaller than n: encrypt a symmetric key, not a file (*hybrid*: random AES key and IV, message under AES-256-CBC, key under the recipient's RSA public key). Textbook RSA is deterministic and malleable: a 4-digit PIN is found by encrypting all 10^4 candidates and comparing; real systems use randomised padding (OAEP).

From the TP: OpenSSL mission 3

`openssl genrsa -out k.pem 512` writes an *unencrypted* private key (anyone who reads the file owns it); `openssl rsa -in k.pem -text -noout` shows n, e, d, p, q; `-pubout` extracts the public key, which may be stored in plaintext; `-aes256` at generation encrypts the key file under a passphrase with AES-256-CBC; `pkeyutl -encrypt -pubin` works on a short file and fails on a large one. Factoring the 50-bit `otherSmall_pub.pem`: extract n with `-pubin -text`, trial-divide to $\sqrt{n}$, compute d.

Go deeper: the details an examiner asks about

- e must be coprime to φ or d does not exist; common $e = 65537 = 2^{16}+1$.

- Primes "similar in magnitude": a small prime is found by trial division; primes too close fall to Fermat's method $n = a^2 - b^2$; $p = q$ makes n a square.
- The slide says "modulo a prime"; precisely, modulo $n = pq$.

Hard questions, answered

Q. Why is combining AES and RSA better than RSA alone for a long message?
A. RSA is slow and bounded by n; AES is fast and unbounded. RSA protects only the 32-byte key; AES does the bulk; no pre-shared secret is needed.

Q. Eve knows (n, e) and the ciphertext of a 4-digit PIN. Attack?
A. Encrypt all 10^4 PINs under (n, e) and compare: textbook RSA is deterministic. Fix: OAEP padding.

5.4 Hash Functions and Passwords

Definition 5.4.1 (Cryptographic hash)**.** H maps arbitrary-length input to a fixed-length digest, fast, lossy, with a tiny input change producing a very different digest. Requirements: **preimage resistance** (given y, hard to find x with $H(x) = y$), **second-preimage resistance** (given x, hard to find $x' \neq x$ with $H(x') = H(x)$), **collision resistance** (hard to find *any* $m_1 \neq m_2$ with equal digests). Collisions exist (finite outputs); the question is the work to find one: generic preimages cost $\approx 2^n$, collisions $\approx 2^{n/2}$ (birthday bound). Collision resistance implies second-preimage resistance; neither implies preimage resistance. MD5 and SHA-1 have practical collisions; SHA-2 (224/256/384/512) is current.

Theorem 5.4.2 (A hash alone gives no authenticity)**.** *If Alice sends $(m, H(m))$ over a channel Eve controls, Eve can replace it by $(m', H(m'))$ and Bob's check passes.*

Proof. H is public and keyless; Eve computes $H(m')$ herself. Integrity against an adversary needs a secret: a MAC key or a signing key. □

Password storage

Store H(password), not the password; on login hash the input and compare. A stolen digest is harder to use than plaintext, but *preimage resistance assumes an unknown high-entropy input*: a password from a small dictionary is found by enumeration however good H is. **Dictionary attack**: precompute digests of common passwords; unsalted, the same password gives the same digest for every user, so one table serves all accounts and equal digests reveal shared passwords. **Salt**: a random per-account value stored beside the digest, combined with the password before hashing; it need not be secret, since its job is to make each account's computation *different*: no shared tables, one campaign per user. Salts do not strengthen a weak password; *slow* password hashes (PBKDF2, bcrypt, scrypt, Argon2) raise the per-guess cost; OpenSSL's `-pbkdf2` is such a derivation.

Go deeper: the details an examiner asks about

- Hashing on the client before sending does not remove server-side hashing: the client hash *is* the password and replays.
- "MD5 is broken" matters for signatures (an attacker crafting two documents with one digest) more than for integrity checksums against accidental corruption, where a second preimage is still needed.

Hard questions, answered

Q. Two users share a password. Unsalted vs salted database?
A. Unsalted: identical digests; cracking one cracks both; precomputed tables apply everywhere. Salted: different digests; separate work per account.

Q. Why is $2^{n/2}$ enough for collisions?
A. Among k random digests the number of pairs is $\approx k^2/2$; the expected number of equal pairs reaches 1 when $k^2/2 \approx 2^n$, i.e. $k \approx 2^{n/2}$ (birthday paradox).

5.5 MACs and Digital Signatures

Definition 5.5.1 (MAC; signature)**.** A **message authentication code** uses a shared key: Alice sends $(m, t = \mathrm{MAC}(K, m))$; Bob recomputes and compares; Eve without K cannot forge a valid tag. Constructions: the last CBC block of $E_K(m)$ (sender needed key and message together; expensive), or hash-based: hash plus symmetric encryption of the digest, or hashing message with a secret value; the hash must be cryptographic. MAC gives integrity and shared-key authentication, *no* confidentiality, *no* non-repudiation (both parties hold K; Bob could have made the tag). A **digital signature** uses the sender's private key: $s = \mathrm{Sign}(\mathrm{priv}, H(m))$, verified with the public key; gives origin authentication, integrity and signatory **non-repudiation** (NIST FIPS 186-4); algorithms RSA, DSA, ECDSA; no pre-shared key.

Reading the formula

Why sign the hash: a 15 MB document under 1024-bit RSA would need thousands of slow operations, and signing raw chosen messages exposes the private key to algebraic attacks; the digest is short and fixed-size, so the signature size is independent of the message. The scheme then needs collision resistance: if Eve finds m_1, m_2 with one digest and gets m_1 signed, the signature validates m_2. *The property table*: encryption C only; MAC I + authentication; signature I + authentication + non-repudiation; none gives availability. Confidentiality and authenticity are separate: secrets sent anonymously (C without A), public information from a trusted source (A without C), secure communication (both).

Hard questions, answered

Q. A confidential contract Bob cannot later deny receiving from Alice. Primitives and order?

A. Sign with Alice's private key over $H(m)$ (authenticity, non-repudiation), then encrypt for Bob (hybrid). Sign first so the signature binds the plaintext.

Q. Is MAC $= H(K\|m)$ safe?
A. Not with Merkle–Damgård hashes: length extension lets Eve compute $H(K\|m\|\text{pad}\|m')$ without K. HMAC (or SHA-3) fixes it.

Q. Why can a MAC not be used as evidence in court?
A. The verifier holds the same key as the signer and could have produced the tag; a third party cannot attribute it. Only a signature attributes.

5.6 Certificates and PKI

The recursion: to verify Alice's signature we need her public key; she cannot prove remotely that a key is hers by signing it with that key. Someone else signs a message binding Alice's identity to her key (Bob); Bob's key is vouched for by Clara, Clara's by Don; Don's key "was shipped to us" and is trusted implicitly.

Definition 5.6.1 (Certificate, CA, chain, PKI)**.** A **certificate** contains an identity and a public key and is signed by someone else; each signer is a **certificate authority**; the implicitly trusted one is a **root CA**; the signatures form a **chain of trust** (Alice ← Bob "CA" ← Clara "CA" ← Don "root"). Obtaining one: generate a key pair (private key secret forever); send the public key and identity to the CA as a **certificate signing request** (everything but the signature) with proof of identity; the CA verifies and signs. Format X.509 (RFC 5280). **PKI** (RFC 4949): the hardware, software, people, policies and procedures to create, manage, store, distribute and *revoke* certificates; it includes a **trust store**, the list of root CAs and keys chosen by the OS or application vendor.

Interpretation & discussion

The base case is trust by *distribution*, not cryptography: root keys arrive with the OS. Whoever controls the trust store controls everything above it: organisations add a private root to sign internally; malware that adds a root can sign a certificate for *any* site and mount man-in-the-middle attacks on every HTTPS connection from that machine; some "security" appliances do exactly this to inspect encrypted traffic, which the course calls stupid and dangerous because one compromised CA key breaks confidentiality for everyone behind it. Verification by a browser: the server certificate's signature by the intermediate, the intermediate's by a trusted root, validity dates, name match, revocation status; only then is the server's public key used.

Hard questions, answered

Q. Why can't Alice self-sign to prove her key?
A. Verifying needs her public key, the thing in question; a self-signature only shows the key pair's holder signed. Trust must enter from outside.

Q. CSR vs the CA generating the pair: difference?
A. With a CSR the private key never leaves Alice; the CA attests only the public key. A CA-generated pair means the CA holds the private key, de-

stroying non-repudiation.

Q. A root CA is added to one laptop by malware. What can the attacker do and what remains safe?
A. MITM every HTTPS connection from that laptop (read and alter). Other machines are unaffected; pinned certificates or fingerprint checks may detect it.

5.7 Steganography

Cryptography hides *meaning*; steganography hides *existence*. Simmons (1983) framed it as the *Prisoners' Problem*: Alice and Bob communicate through a warden; the carriers are *stego objects*, the hidden data the *payload*; a *passive* warden only observes (detection problem), an *active* warden alters the objects (robustness problem). Techniques: null ciphers (third letters of each word: "Send Lawyers, Guns, and Money"; first letters of paragraph one and second letters of paragraph two in the WWI telegram: "PERSHING SAILS FROM N.Y. JUNE 1"); text, image, audio, video, network and email steganography; **LSB**: overwrite the least significant bit of each pixel or sample with payload bits, changing each value by at most 1/255 (1 bit per byte of capacity; detectable by LSB statistics; destroyed by recompression). First use: Greece, c. 440 BC. Encrypt the payload first so that detection does not reveal meaning.

Hard questions, answered

Q. Hide 'A' (`01000001`) in pixels `10110010, 10110011, 10110010, 10110011, 10110010, 10110011, 10110010, 10110011`.
A. `10110010, 10110011, 10110010, 10110010, 10110010, 10110010, 10110010, 10110011`.

Q. Which warden defeats LSB trivially and how?
A. Active: re-encode (JPEG, blur, resample); the LSB plane is destroyed, the picture survives.

Summary: Public-key cryptography, integrity, trust

- $N(N-1)/2$ vs $2N$ keys; trapdoor one-way functions; authenticity of public keys becomes the problem.
- RSA: $n = pq$, φ, $ed \equiv 1$; correct by Euler; secure by factoring; hybrid with AES; padding against determinism.
- Hash: preimage / second-preimage / collision ($2^{n/2}$); keyless, so no authenticity; passwords: salt against shared tables, slow hashes against speed.
- MAC (shared key: I + auth) vs signature (private key over $H(m)$: + non-repudiation); encryption gives C only.
- Certificates bind identity to key by a CA's signature; chain to a distributed root; trust store is the root of everything; PKI includes revocation.
- Steganography hides existence; LSB; passive vs active warden.